University of Belgrade
School of Electrical Engineering
Department of Electronics

Fundamentals of Digital Electronics
- 13E042OD -

Homework Report

Hardware Design Using *VHDL* Traffic-Light Controller

Homework completed by:

Full name:	**Student ID:**
Viktoria Đorđević	**2024/0021**
Mira Vukmirović	**2024/0105**

Hardware Design in VHDL

The traffic-light controller is implemented as a finite-state machine. The code defines the type state_type, which contains the states redRed1, redGreen, redRed2, redYellowRed, greenRed, and yellowRed.

At a clock frequency of 8 Hz, one second corresponds to eight clock periods. Constants in the code therefore convert the duration of each state into a number of clock cycles. For example, the greenRed state lasts 22 seconds, so its duration is defined as T_GREEN_RED = 22 * CLK_HZ = 176.

```
-- Sequential logic: stores the state and counts clock cycles.
process(clk, reset)
begin
    if reset = '1' then
        state   <= redRed1;
        counter <= T_RED_RED_1;

    elsif rising_edge(clk) then
        if counter = 1 then
            case state is
                when redRed1 =>
                    state   <= redGreen;
                    counter <= T_RED_GREEN;

                when redGreen =>
                    state   <= redRed2;
                    counter <= T_RED_RED_2;

                when redRed2 =>
                    state   <= redYellowRed;
                    counter <= T_RED_YELLOW_RED;

                when redYellowRed =>
                    state   <= greenRed;
                    counter <= T_GREEN_RED;

                when greenRed =>
                    state   <= yellowRed;
                    counter <= T_YELLOW_RED;

                when yellowRed =>
                    state   <= redRed1;
                    counter <= T_RED_RED_1;
            end case;
        else
            counter <= counter - 1; -- Not yet time to enter the next state
        end if; -- Stay in this state and decrement the counter by one cycle.
    end if;
end process;
```

An auxiliary signal, counter, of type integer is used to measure each state's duration.

Each state corresponds to one phase of the traffic-light cycle. When the active-high reset is asserted, the machine enters the initial state, redRed1. The output signals rc, gc, yc, rp, and gp depend only on the current state of the machine. This part is therefore implemented as combinational logic in a separate process.

The active-high reset signal returns the machine to the initial state, redRed1, and sets the counter to that state's duration. The counter value is checked on every rising edge of the clock. If counter = 1, the current state has ended: the machine moves to the next state and loads the counter with the duration of that new state. If the counter has not yet reached 1, the state remains unchanged and the counter is decremented by one: counter <= counter - 1;

The remaining-time display is implemented separately for the greenRed state, when vehicles have a green light and pedestrians have a red light. In this state, the outputs time_d1 and time_d2 show the number of seconds remaining until the state ends.

```
120
121     -- Show remaining time only in the greenRed state.
122     -- In all other states, 1111 blanks the bcd_to_7seg display.
123     process(state, counter)
124         variable sec_left : integer range 0 to 22;
125         variable tens     : integer range 0 to 9;
126         variable ones     : integer range 0 to 9;
127     begin
128         if state = greenRed then
129             sec_left := (counter + CLK_HZ - 1) / CLK_HZ;
130             tens     := sec_left / 10;
131             ones     := sec_left mod 10;
132
133             time_d2 <= std_logic_vector(to_unsigned(tens, 4));
134             time_d1 <= std_logic_vector(to_unsigned(ones, 4));
135         else
136             time_d2 <= "1111";    -- Display off
137             time_d1 <= "1111";
138         end if;
139     end process;
140
141 end Behavioral;
```

The signal time_d1 represents the units digit, and time_d2 represents the tens digit. Both signals are of type std_logic_vector(3 downto 0).

At the start of the traffic-light controller simulation, reset is set to logic 1 because it is active high. Reset remains asserted for three clock periods:

reset <= '1'; wait for 3 * C_CLK_PERIOD;

This ensures that the state machine is placed in its initial state, redRed1. Reset is then deasserted by setting it to logic 0: reset <= '0';

After reset is deasserted, the simulation continues for another 40 seconds. This is long enough to show all traffic-light states, since a complete operating cycle lasts: 2 s + 9 s + 2 s + 1 s + 22 s + 1 s = 37 s.

The figures below show the traffic-light states during the VHDL simulation.

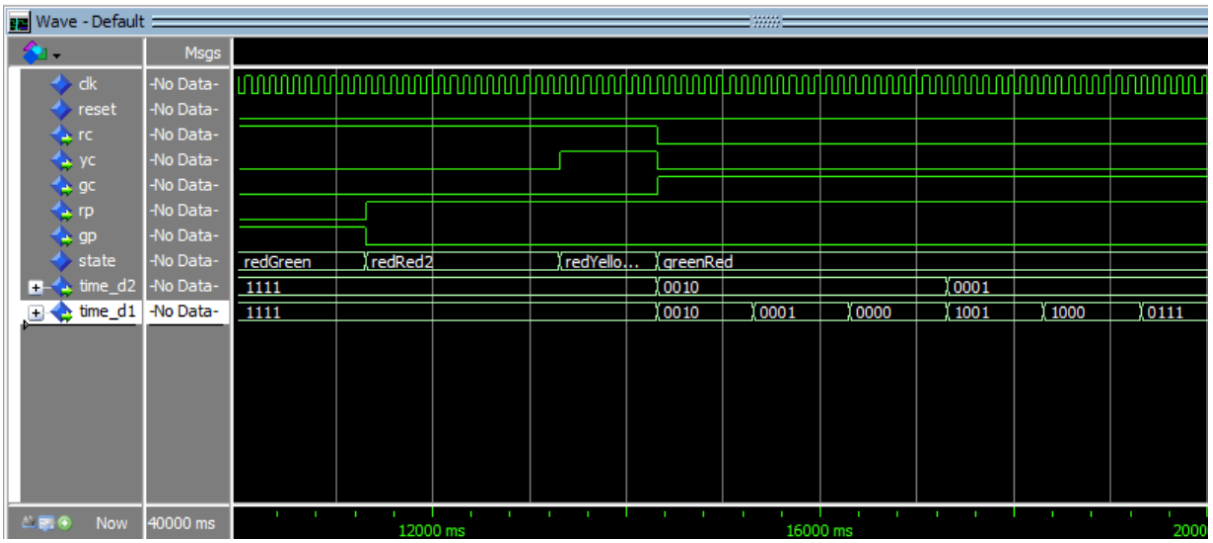


Figure 1 - Start of the traffic-light controller's operating cycle

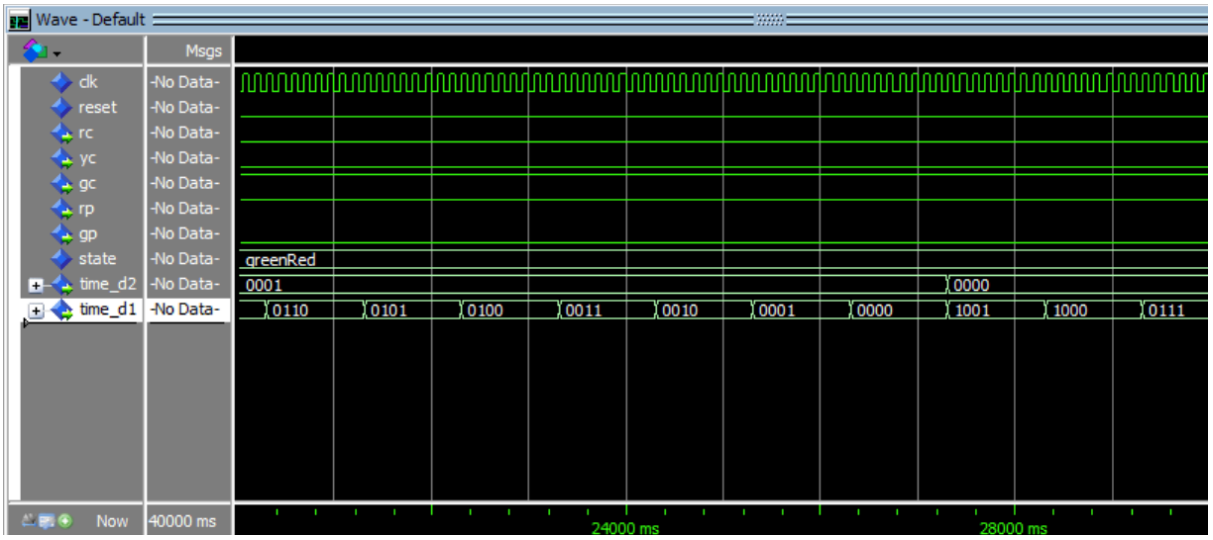


Figure 2 - The greenRed state and the remaining-time countdown

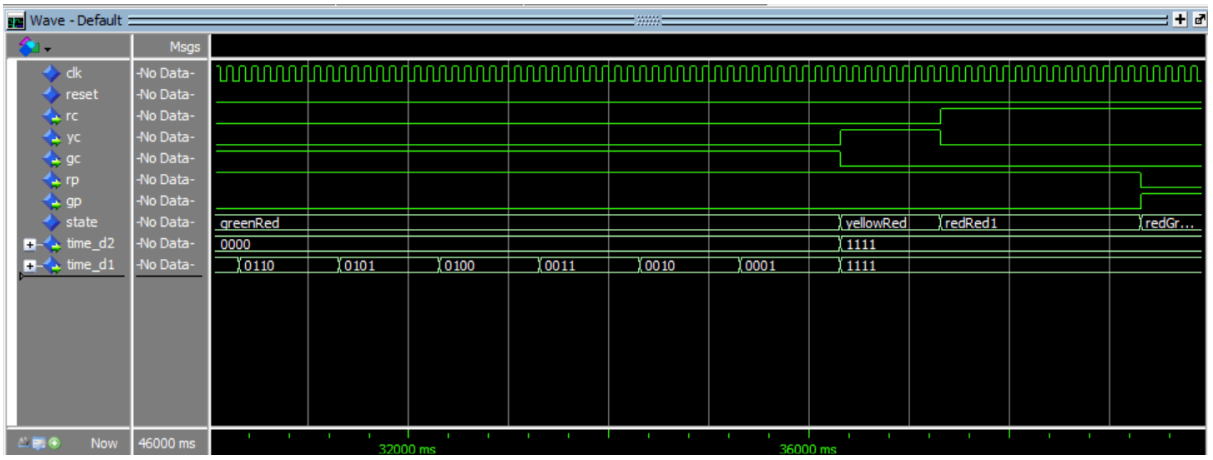


Figure 3 - Transition from greenRed to yellowRed and return to redRed1

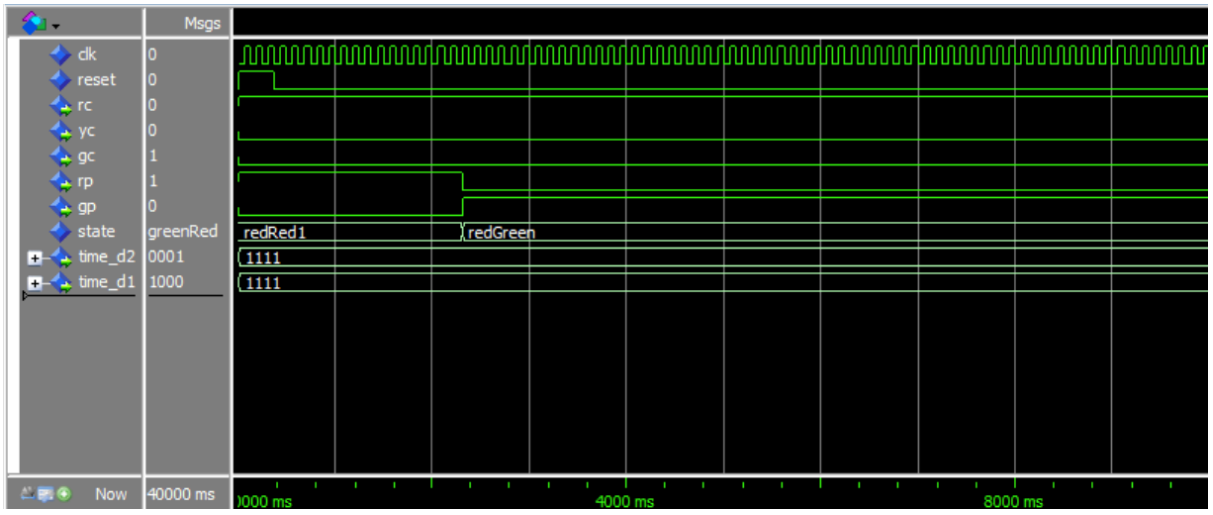


Figure 4 - Start of the next traffic-light cycle

The bcd_to_7seg module converts a single BCD digit into a code for a seven-segment LED display. The input signal digit is of type std_logic_vector(3 downto 0), and the output signal display is of type std_logic_vector(6 downto 0).

If the input is not a digit from 0 to 9, the output is set to display <= "0000000", turning off all display segments. The traffic-light controller also uses this behavior: in all states except greenRed, the BCD converters receive "1111", so the displays are blank.

The conversion is implemented using a case statement. Since this is a combinational circuit, the display output changes as soon as the digit input changes. For each valid BCD digit, from 0 to 9, a corresponding seven-bit pattern turns on the required display segments. If the input is not a valid BCD digit, i.e., if it is between 10 and 15, the when others branch is selected. The output is then set to "0000000", meaning that all segments are off:

```
    process(digit)
    begin
        case digit is
            when "0000" => display <= "0111111"; -- 0
            when "0001" => display <= "0000110"; -- 1
            when "0010" => display <= "1011011"; -- 2
            when "0011" => display <= "1001111"; -- 3
            when "0100" => display <= "1100110"; -- 4
            when "0101" => display <= "1101101"; -- 5
            when "0110" => display <= "1111101"; -- 6
            when "0111" => display <= "0000111"; -- 7
            when "1000" => display <= "1111111"; -- 8
            when "1001" => display <= "1101111"; -- 9
            when others => display <= "0000000"; -- Not a BCD digit; display off
        end case;
    end process;

end Behavioral;
```

The figure below shows the seven-segment decoder's behavior in the VHDL simulation.

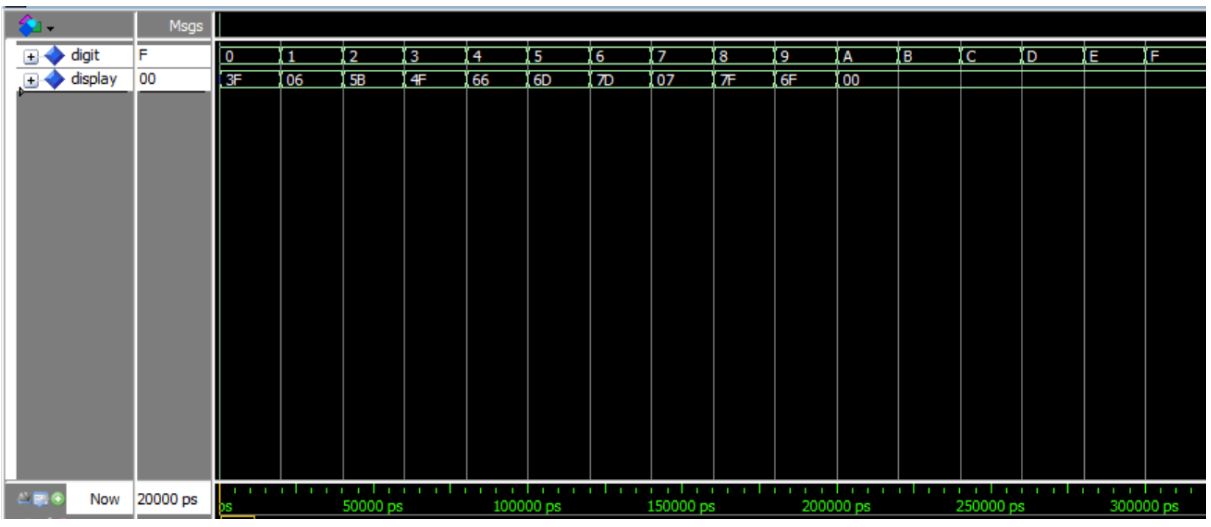


Figure 5 - Timing diagram of the bcd_to_7seg component

The auxiliary signals time_d1_s and time_d2_s carry the units and tens digits from the traffic_light module to the corresponding BCD-to-7seg converters. The outputs display0 and display1 thus provide signals suitable for directly displaying the remaining time on seven-segment LED displays.

The file traffic_light_system.vhd implements the top-level module, which connects the previously designed components into a single system. It instantiates the traffic_light module, which generates the traffic-light signals and the BCD digits of the remaining time, as well as two bcd_to_7seg modules, which convert those BCD digits into seven-segment display codes. The figures below show the complete system operating in the VHDL simulation.

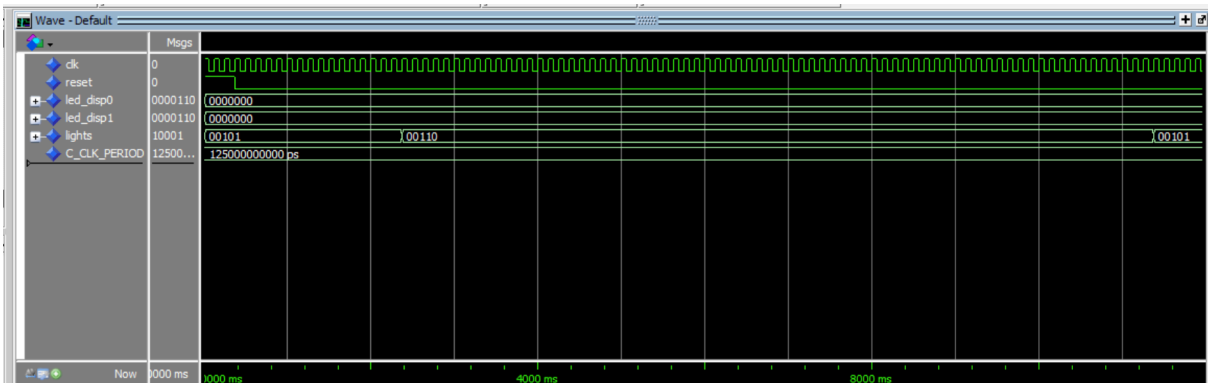


Figure 6 - Start of the complete traffic_light_system simulation

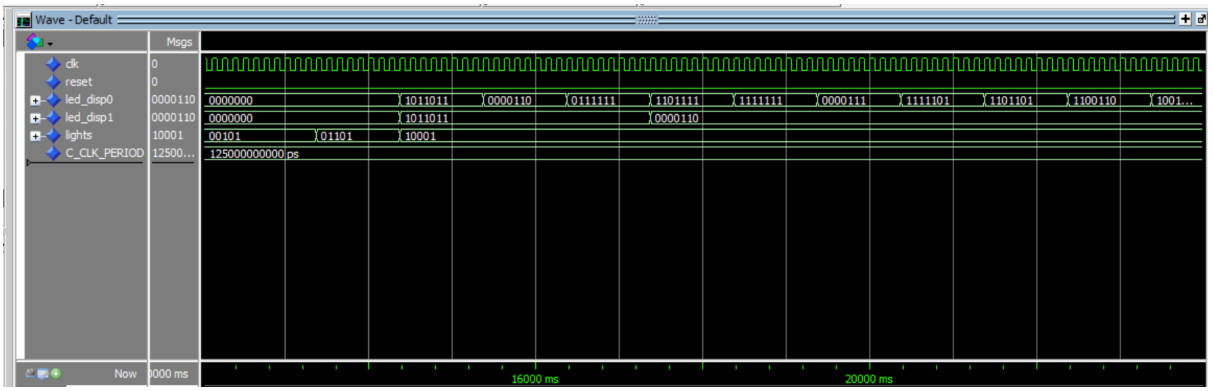


Figure 7 - Remaining-time countdown in the greenRed state of the complete system

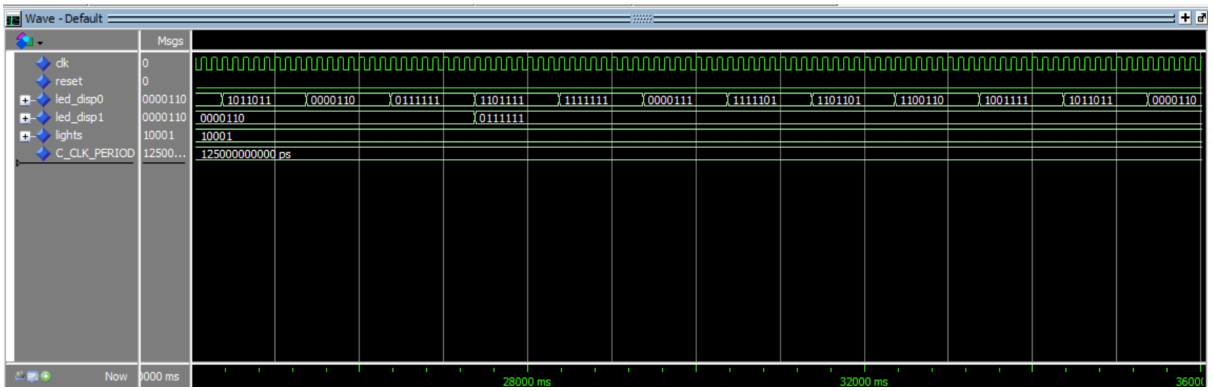


Figure 8 - Complete system: greenRed to yellowRed, then back to redRed1

Simulation Results in Logisim

Recording of the simulation in Logisim:

Logisim_ main of traffic_light_simulator1 2026-06-24 19-44-52.mp4